The final method, which finishes the consumer contract interaction, takes a Pact class argument. In this example, we're using the V4 Pact file format. Bringing it all together, we've created a consumer and provider interaction within a function that will generate a contract and form the mock within our consumer test, as shown in figure 5.6.

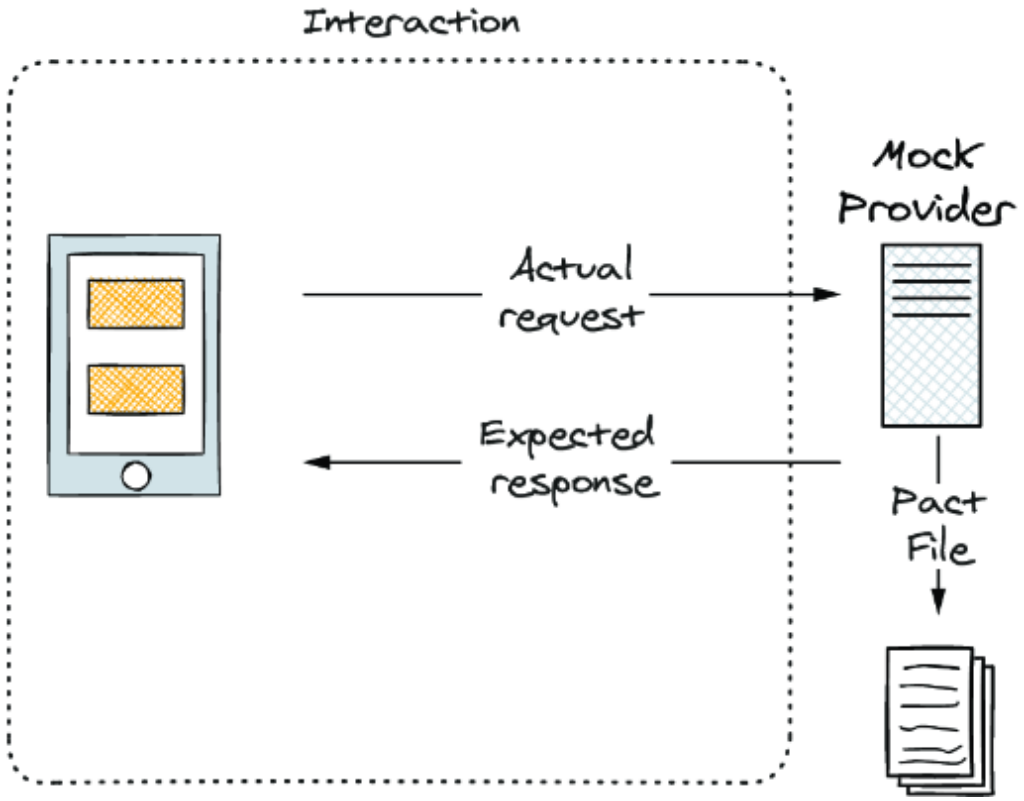


Figure 5.6 Pact interaction contains the mobile app's actual request intercepted and the expected response returned from the mock server. Finally, the interactions are converted to a Pact file.

Looking at the complete picture, the `createPact` function is annotated with the consumer and provider related to the specific request. The function takes `PactDslWithProvider` as an argument and returns the V4 Pact file format as the output. The body of the expected response is defined as a `DslPart` object. Finally, the builder chains all the request and response details needed to set up the contract, which maps directly to how the request is made in the code. The full version of the code is shown in the following listing, which shows the test and how it calls this function we've created.